

13. Bölüm

Yapılar ve Birlikler

13.1 Yapılar

Yapısal programlamaya adını veren yapı (**structure**), türetilmiş (**derived**) veya kullanıcı tanımlı (**user defined**) bir veri tipidir. Farklı tiplerdeki elemanları bir arada gruplandıran özel bir veri tipi tanımlamak için **struct** anahtar kelimesini kullanırız. Yapı bir veya birden fazla ilkel veri tipinin (**char**, **int**, **long**, **float**, **double**, ...) ve bu tiplere ait dizilerin bir araya gelmesiyle oluşturulan yeni veri tipleridir.

Diziler, aynı tipte elemanlardan oluşmasına rağmen, yapılar farklı dipte elemanların bir araya gelmesiyle oluşabilir. Aşağıda sözde kodu verilmiştir;

```
struct yapı-kimliği {  
    veri-tipi yapı-elemanı-kimliği1;  
    veri-tipi yapı-elemanı-kimliği1;  
    ...  
} [değişken-kimliği1][, değişken-kimliği2];
```

Örnek olarak **adı**, **soyadı**, **numarası**, **yaşı**, **cinsiyeti** gibi farklı öğeler ile bir **ogrenci** yapısı tanımlanabilir;

```
struct ogrenci {  
    char adi[50];  
    char soyadi[0];  
    int yas;  
    int cinsiyet;  
} ogrenci1;
```

Yukarıda yapılan bu tanımlama ile **struct ogrenci** veri tipine **ogrenci1** kimliğine sahip bir değişken tanımlanmıştır. Yapı tanımlandıktan sonra aşağıdaki sözde kodda verildiği gibi yeni değişkenler tanımlanabilir;

```
struct yapı-kimliği değişken-kimliği;
```

Yukarıda tanımlanan **ogrenci** yapısından **ogrenci2** ve **ogrenci3** kimlikli birçok değişken tanımlanabilir;

```
struct ogrenci ogrenci2,ogrenci3;
```

Yapının elemanlarına tanımlama sırasında **başlangıç değeri** (**initial value**) verilebilir yani yapı değişkeni bazı değerlerle **başlatılabilir** (**initialize**). Bunun için diziler gibi süslü parantez kullanılır;

```
struct ogrenci ogrenci4={"Deniz", "AK", 35, 2};  
struct ogrenci ogrenci5={"Damla", "YILDIZ", 28, 2};
```

Bilgi

Bir yapının içinde bir başka yapı da tanımlanabilir. Yani iç içe yapılar (**nested structs**) tanımlanabilir.

13.2 Yapı Elemanlarına Erişim

Tanımlanan yapı değişkenleri üzerinden her bir alamana **nokta işleci** (**dot operator**) erişilir. Bu işlecin önceliği parantez ile aynıdır. Atama işleci (=) bir yapıyı doğrudan kopyalamak için kullanılabilir. Ayrıca, bir yapının üyesinin değerini başka birine atamak için atama işlecini de kullanabiliriz.

```
#include <stdio.h>  
#include <string.h>  
struct ogrenci {  
    char adi[50];  
    char soyadi[50];  
    int yas;  
    int cinsiyet;  
} ogrenci1;  
int main() {  
    /* ogrenci1 yapısı başlatılmadığından aşağıdaki gibi değerleri güncelleniyor: */
```

```

strcpy(ogrenci1.adi, "Ilhan");
strcpy(ogrenci1.soyadi, "OZKAN");
ogrenci1.yas=50;
ogrenci1.cinsiyet=1;
printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n", ogrenci1.adi, ogrenci1.soyadi,
    → ogrenci1.yas, ogrenci1.cinsiyet);
struct ogrenci ogrenci2={"Deniz","AK",35,2}; /* yapı bazı değerlerle başlatıldı. */
printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n",ogrenci2.adi, ogrenci2.soyadi,
    → ogrenci2.yas, ogrenci2.cinsiyet);
struct ogrenci ogrenci3=ogrenci2; // yapı kopyalama
printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n", ogrenci3.adi, ogrenci3.soyadi,
    → ogrenci3.yas, ogrenci3.cinsiyet);
return 0;
}

```

13.3 Yapı Göstericileri

Yapı göstericileri, tıpkı diğer değişkenlere gösterici tanımladığımız gibi tanımlayabiliriz. Gösterici üzerinden yapı değişkenlerine **dolaylı işleç (indirection operator)** yani (->) ile erişilir. Bu nokta işleci ile aynı önceliklidir.

Yapılar ve göstericileri; veri tabanları, dosya yönetim uygulamaları ve ağaç ve bağlı listeler gibi karmaşık veri yapılarını işlemek gibi farklı uygulamalarda kullanılır.

```

#include <stdio.h>
#include <string.h>
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet;
};
int main() {
    struct ogrenci ogrenci1;
    strcpy(ogrenci1.adi, "Ilhan");
    strcpy(ogrenci1.soyadi, "OZKAN");
    ogrenci1.yas=50;
    ogrenci1.cinsiyet=1;
    struct ogrenci* ogrenciGosterici;
    ogrenciGosterici=&ogrenci1;
    printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d", ogrenciGosterici->adi,
        → ogrenciGosterici->soyadi, ogrenciGosterici->yas,ogrenciGosterici->cinsiyet);
    return 0;
}

```

13.4 Yapılar ve Fonksiyonlar

Sıradan dizilere benzer şekilde bir yapı dizisi tanımlayabiliriz. Ayrıca yapı değişkenini bir fonksiyona parametre olarak gönderebilir ve bir fonksiyondan bir yapı döndürebilirsiniz;

```

#include <stdio.h>
#include <string.h>
enum cinsiyet {BELIRTIOMEMIS,KADIN,ERKEK};
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet;
};
struct ogrenci yeniOgrenci();
void ogreciYaz(struct ogrenci);
int main() {
    struct ogrenci ogrenciler[30];
    struct ogrenci o=yeniOgrenci();
    ogreciYaz(o);
}

```

```

    return 0;
}
struct ogrenci yeniOgrenci() {
    struct ogrenci yeni={"Ilhan", "Ozkan", 50, ERKEK};
    return yeni;
}
void ogrenciYaz(struct ogrenci pOgrenci) {
    printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n", pOgrenci.adi, pOgrenci.soyadi,
        ↪ pOgrenci.yas, pOgrenci.cinsiyet);
}

```

Yapı elemanları değer tipler olduğundan, yapılara ait göstercileri parametre olarak kullanmak, olabilecek değişiklikleri aktarmak için üstünlük sağlar;

```

#include <stdio.h>
#include <string.h>
enum cinsiyet {BELIRTILMEMIS, KADIN, ERKEK};
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet; //1:erkek,2:Kadın,0:Belirtmiyor
};
struct ogrenci yeniOgrenci();
void ogrenciOku(struct ogrenci*);
void ogrenciYaz(struct ogrenci*);
int main() {
    struct ogrenci o=yeniOgrenci();
    ogrenciYaz(&o);
    ogrenciOku(&o);
    ogrenciYaz(&o);
    return 0;
}
struct ogrenci yeniOgrenci() {
    struct ogrenci yeni={"Ilhan", "Ozkan", 50, ERKEK};
    return yeni;
}
void ogrenciOku(struct ogrenci* pOgrenci) {
    printf("Adı Giriniz:");
    scanf("%s", pOgrenci->adi);
    printf("Soyadı Giriniz:");
    scanf("%s", pOgrenci->soyadi);
    printf("Yaş Giriniz:");
    scanf("%d", &(pOgrenci->yas));
    printf("Cinsiyet Giriniz (0-1-2):");
    scanf("%d", &(pOgrenci->cinsiyet));
}
void ogrenciYaz(struct ogrenci* pOgrenci) {
    printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n", pOgrenci->adi, pOgrenci->soyadi,
        ↪ pOgrenci->yas, pOgrenci->cinsiyet);
}

```

13.5 Anonim Yapılar

Anonim yapı, kimlik veya **takma isim** (**typedef**) ile tanımlanmayan bir yapı tanımıdır. Genellikle başka bir yapının içine yerleştirilir. C11 sürümü ile kullanılmaya başlanan bu özelliğin aşağıda sıralanan üstünlükleri vardır;

- ◊ **Esneklik (flexibility)**: Anonim yapılar, verilerin nasıl temsil edildiği ve erişildiği konusunda esneklik sağlayarak daha dinamik ve çok yönlü veri yapılarına olanak tanır.
- ◊ **Kolaylık (convenience)**: Bu özellik, farklı veri tiplerini tutabilen bir değişkenin kompakt bir şekilde temsil edilmesine olanak tanır.
- ◊ **Başlatma Kolaylığı (easy of initialization)**: Yapı değişkenine ilişkin ek kimliklendirme yapılmadan ilk değer verilmeleri ve kullanılmaları daha kolay olabilir.

İç içe yapıların (**nested structs**) bir örneği olan anonim yapılara, **ogrenci** yapımıza doğum tarihini içerecek anonim yapı eklenen örnek, aşağıda verilmiştir;

```
#include <stdio.h>
#include <string.h>
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet; //1:erkek,2:Kadın,0:Belirtmiyor
    struct { //anonim yapı: yapı kimliği yoktur!
        int gun;
        int ay;
        int yil;
    };
};
struct ogrenci yeniOgrenci();
void ogrenciYaz(struct ogrenci);
int main() {
    struct ogrenci o=yeniOgrenci();
    ogrenciYaz(o);
    return 0;
}
struct ogrenci yeniOgrenci() {
    struct ogrenci yeni={"Ilhan","Ozkan",50,1, {1,1,1970}};
    return yeni;
}
void ogrenciYaz(struct ogrenci pOgrenci) {
    puts(pOgrenci.adi);
    puts(pOgrenci.soyadi);
    printf("Yas:%d\n",pOgrenci.yas);
    printf("Cinsiyet:%d\n",pOgrenci.cinsiyet);
    printf("Dogum Tarihi:%d-%d-%d\n",pOgrenci.gun,pOgrenci.ay,pOgrenci.yil);
}
```

13.6 Bileşik Değişmezler

Bileşik değişmezler (**compound literals**), C diline C99 standardı ile eklenmiştir. Yani 1999 yılından beri modern C'nin bir parçasıdır. Özellikle yapı (**struct**) ve dizilere, geçici bir değişken tanımlamadan değer atamak için kullanılır.

```
#include <stdio.h>

struct Nokta {
    int x;
    int y;
};

void ciz(struct Nokta p) {
    printf("Nokta çiziliyor... Koordinatlar: x = %d, y = %d\n", p.x, p.y);
}

int main() {
    // YÖNTEM A: Geleneksel yöntem (Önce değişken tanımla, sonra gönder)
    struct Nokta n1 = {5, 5};
    ciz(n1);

    // YÖNTEM B: Bileşik Sabit (Compound Literal) kullanımı
    // Burada geçici bir değişken ismi tanımlamadan doğrudan veri oluşturuyoruz.
    ciz((struct Nokta){.x = 10, .y = 20});

    // İsterseniz indis belirtmeden de yazabilirsiniz:
    ciz((struct Nokta){30, 40});
    return 0;
}
```

}

Özellik	(int)5.5 (Cast)	(int[]){5, 6} (Literal)
Bellek (Memory)	Sadece CPU kaydedicisinde bir değerdir.	Yığın (stack) ya da veri (data) bellekte statik değişken gibi yer kaplar.
Adres (&)	Adresi alınamaz: &(int)5.5 HATA	Adresi alınabilir: &(int[]){1,2} GEÇERLİ
Değiştirilebilirlik	Değiştirilemez (Sabit bir değer döner).	İçindeki değerler değiştirilebilir.

Tablo 13.1: Bileşik Sabitler ve Tip Dönüşümü Karşılaştırması

13.7 Öz Referanslı Yapılar

Öz referanslı yapı (self-referential struct), öğelerinden bir veya daha fazlası kendi tipindeki göstericilerden oluşan bir yapıdır. Öz referanslı yapılar, bağlantılı listeler ve ağaçlar gibi karmaşık olan **dinamik veri yapıları** (dynamic data structure) içinde yaygın olarak kullanılırlar.

```
#include <stdio.h>
struct ogrenci {
    char adi[10]; char soyadi[10]; int yas;
    struct ogrenci* sonrakiOgrenci; // Kendi veri tipinden bir yapı göstericisi!
};
void ogrecileriYaz(struct ogrenci*);
int main() {
    struct ogrenci ilk={"Ilhan","OZKAN",50,NULL};
    struct ogrenci sonraki={"Ayse","YILMAZ",40,NULL};
    ilk.sonrakiOgrenci=&sonraki;
    struct ogrenci birsonraki={"Tahsin","BULUT",35,NULL};
    sonraki.sonrakiOgrenci=&birsonraki;
    ogrecileriYaz(&ilk);
    return 0;
}
void ogrecileriYaz(struct ogrenci* pIlk) {
    int sayac=0;
    while (pIlk!=NULL) {
        printf("OGRENCI %d: Adı:%s Soyad:%s Yaş:%d\n", ++sayac, pIlk->adi, pIlk->soyadi,
            ↪ pIlk->yas);
        pIlk=pIlk->sonrakiOgrenci;
    }
}
/*Program Çıktısı:
OGRENCI 1: Adı:Ilhan Soyad:OZKAN Yaş:50
OGRENCI 2: Adı:Ayse Soyad:YILMAZ Yaş:40
OGRENCI 3: Adı:Tahsin Soyad:BULUT Yaş:35
*/
```

13.8 Hizalama

C11 standardı ile gelen `_Alignof` işleci, bir veri tipinin bellekte hangi bayt sınırlarına göre hizalanması (alignment) gerektiğini söyler. Örneğin, 4 baytlık bir `int` genellikle bellekte 4'ün katı olan adreslerde saklanır. İşte bu işlecin ve C11 ile gelen `stdalign.h` kütüphanesini kullanan basit bir program;

```
#include <stdio.h>
#include <stdalign.h> // alignof makrosu için (C11)

// Örnek bir yapı (struct)
struct Yapi {
    char c; // 1 bayt
    double d; // 8 bayt
    int i; // 4 bayt
};
```

```

int main() {
    printf("--- Veri Tipleri Hizalama Gereksinimleri ---\n");

    // Temel tiplerin hizalaması
    printf("char hizalamasi      : %zu bayt\n", alignof(char));
    printf("int hizalamasi       : %zu bayt\n", alignof(int));
    printf("double hizalamasi    : %zu bayt\n", alignof(double));

    // Kendi oluşturduğumuz yapının hizalaması
    printf("struct hizalamasi   : %zu bayt\n", alignof(struct Yapi));

    printf("--- Karşılaştırma (Size vs Align) ---\n");
    printf("struct boyutu (sizeof)   : %zu bayt\n", sizeof(struct Yapi));
    printf("struct hizalamasi (align): %zu bayt\n", alignof(struct Yapi));

    return 0;
}
/*Program Çıktısı:
--- Veri Tipleri Hizalama Gereksinimleri ---
char hizalamasi      : 1 bayt
int hizalamasi       : 4 bayt
double hizalamasi    : 8 bayt
struct hizalamasi    : 8 bayt
--- Karşılaştırma (Size vs Align) ---
struct boyutu (sizeof)   : 24 bayt
struct hizalamasi (align): 8 bayt
*/

```

C dilinde anahtar kelime aslında `_Alignof` şeklindedir. Ancak `<stdalign.h>` kütüphanesini eklerseniz, daha okunaklı olan `alignof` ismini kullanabilirsiniz. İşlemciler (CPU), veriye hizalanmış adreslerden (örneğin 4'ün katı adresler) çok daha hızlı erişir. Eğer bir veri yanlış hizalanırsa, CPU o veriyi okumak için iki kat daha fazla çaba sarf edebilir. Bir yapının hizalama gereksinimi, genellikle içindeki en geniş hizalama gerektiren elemana göre belirlenir. Yukarıdaki örnekte `double` (8 bayt) olduğu için tüm yapı muhtemelen 8 baytlık sınıra hizalanacaktır.

Bu programın çalışması için derleyicinizin C11 veya üzerini desteklemesi gerekir. GCC veya Clang kullanıyorsanız `-std=c11` parametresiyle `gcc program.c -o program -std=c11` şeklinde derleyebilirsiniz.

13.9 Yapı Dolgusu

İşlemciler belleğe birer birer değil, genellikle 4 veya 8 baytlık bloklar halinde erişirler. Bu yüzden derleyici, verileri bu blokların başına denk gelecek şekilde yerleştirir. İşte bu süreci ve `_Alignof` etkisini gösteren detaylı bir örnek:

```

#include <stdio.h>
#include <stdalign.h>

// Senaryo 1: Verimsiz dizilim
struct Kotu {
    char a;      // 1 bayt
    // --> 3 bayt DOLGU (Padding) eklenir
    int b;       // 4 bayt
    char c;      // 1 bayt
    // --> 3 bayt DOLGU (Padding) eklenir
};

// Senaryo 2: Verimli dizilim
struct İyi {
    int b;       // 4 bayt
    char a;      // 1 bayt
    char c;      // 1 bayt
    // --> 2 bayt DOLGU (Padding) eklenir
};

```

```
int main() {
    printf("--- 'struct Kotu' Analizi ---\n");
    printf("Boyut (sizeof): %zu bayt\n", sizeof(struct Kotu));
    printf("Hizalama (alignof): %zu bayt\n", alignof(struct Kotu));

    printf("--- 'struct İyi' Analizi ---\n");
    printf("Boyut (sizeof): %zu bayt\n", sizeof(struct İyi));
    printf("Hizalama (alignof): %zu bayt\n", alignof(struct İyi));

    return 0;
}
/*Program Çıktısı:
--- 'struct Kotu' Analizi ---
Boyut (sizeof): 12 bayt
Hizalama (alignof): 4 bayt
--- 'struct İyi' Analizi ---
Boyut (sizeof): 8 bayt
Hizalama (alignof): 4 bayt
*/
```

Bilgisayarın belleğini 4 baytlık (32-bit bir sistemde) kutular olarak düşün. `int` verisi her zaman 4'ün katı olan bir adreste başlamak ister.

`struct Kotu` için hesaplama:

1. `char a`, 0. adrese yerleşir.
2. Sıradaki `int b`, 4. adreste başlamak zorundadır. Bu yüzden 1, 2 ve 3. adresler boş bırakılır (**padding**).
3. `int b`, 4-7 adreslerini kaplar.
4. `char c`, 8. adrese yerleşir.
5. Yapının toplam boyutu, en büyük elemanın (`int`) hizalamasının katı olmalıdır. Bu yüzden sonuna 3 bayt daha eklenir.
6. Bu durumda

$$\text{Toplam Boyut} = 1(\text{char}) + 3(\text{pad}) + 4(\text{int}) + 1(\text{char}) + 3(\text{pad}) = 12 \text{ bayt}$$

olur

C dilinde **yapı dolgusu** (**structure padding**), işlemci (CPU) mimarisi ile belirlenir. Dolgu işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. Bunun nedeni 32 veya 64 bitlik bir bilgisayarda işlemcinin tek seferde bellekten 4 bayt okumasından kaynaklanmaktadır.

`struct İyi` için hesaplama:

- ◇ `int b`, 0-3 adreslerini kaplar.
- ◇ `char a`, 4. adrese yerleşir.
- ◇ `char c`, 5. adrese yerleşir. Toplam boyutu 4'ün katına tamamlamak için sona sadece 2 bayt dolgu eklenir.
- ◇ Bu durumda

$$\text{Toplam Boyut} = 4(\text{int}) + 1(\text{char}) + 1(\text{char}) + 2(\text{pad}) = 8 \text{ bayt}$$

olur.

Bilgi

`_Alignof` bize bir tipin "hizalama kuralını" söylerken, bu kurala uymak için derleyicinin aralara attığı boşluklara dolgu (**padding**) diyoruz. **Büyük sistemlerde veya gömülü cihazlarda**, değişkenleri büyükten küçüğe doğru (örneğin önce `double`, sonra `int`, en son `char`) sıralamak bellekten **ciddi tasarruf sağlar**.

Aşağıdaki yapı örneklerini detaylı inceleyelim;

```
#include <stdio.h>
struct yapi1 {
    char a;
    char b;
    int c;
};
struct yapi2 {
```

```

    char d;
    int e;
    char f;
};
int main() {
    printf("char bellek miktarı: %d\n", sizeof(char));
    // char bellek miktarı: 1
    printf("int bellek miktarı: %d\n", sizeof(int));
    // int bellek miktarı: 4
    printf("-----\n");
    printf("Yapılardaki 2 adet char ve 1 adet int olması gereken bellek miktarı : %d\n",
        ↪ 2*sizeof(char)+sizeof(int));
    // yapılar için olması gereken bellek miktarı: 6
    printf("yapi1 için bellekte ayrılan miktar: %d\n", sizeof(struct yapı1));
    // yapı1 için bellekte ayrılan miktar: 8
    printf("yapi2 için bellekte ayrılan miktar: %d\n", sizeof(struct yapı2));
    // yapı2 için bellekte ayrılan miktar: 12
    return 0;
}
/* Program Çıktısı:
char bellek miktarı: 1
int bellek miktarı: 4
-----
Yapılardaki 2 adet char ve 1 adet int olması gereken bellek miktarı : 6
yapi1 için bellekte ayrılan miktar: 8
yapi2 için bellekte ayrılan miktar: 12
*/

```

Verilen örnekte aynı bellek miktarına sahip elemanlar için yapının bütününe bakıldığında farklı miktarda bellek ayrıldığı anlaşılmaktadır. Dolgu (**padding**) işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. **Kod içerisindeki tanımlanacak tüm yapılar için bunu engellemenin yolu `#pragma pack(1)` ön işlemci yönergesini (**preprocessor directive**) kaynak kodun başına eklemektir;** Eğer yalnızca belirlenen bir yapı için bunun yapılması isteniyorsa yapı tanımına `__attribute__((packed))` özelliği eklenir.

```

#include <stdio.h>
#pragma pack(1) // Tüm yapılarda dolgu yapılsın!
struct yapı1 {
    char a;
    char b;
    int c;
};
struct __attribute__((packed)) yapı2 { //Sadece Bu yapıda dolgu yapılsın!
    char a;
    int b;
    char c;
};
int main() {
    printf("char bellek miktarı: %d\n", sizeof(char));
    // char bellek miktarı: 1
    printf("int bellek miktarı: %d\n", sizeof(int));
    // int bellek miktarı: 4
    printf("-----\n");
    printf("Yapılardaki 2 adet char ve 1 adet int olması gereken bellek miktarı: %d\n",
        ↪ 2*sizeof(char)+sizeof(int));
    // yapılar için olması gereken bellek miktarı: 6
    printf("yapi1 için bellekte ayrılan miktar: %d\n", sizeof(struct yapı1));
    // yapı1 için bellekte ayrılan miktar: 6
    printf("yapi2 için bellekte ayrılan miktar: %d\n", sizeof(struct yapı2));
    // yapı1 için bellekte ayrılan miktar: 6
    return 0;
}
/* Program Çıktısı:

```



```
char bellek miktarı: 1
int  bellek miktarı: 4
-----
Yapılardaki 2 adet char ve 1 adet int olması gereken bellek miktarı: 6
yapi1 için bellekte ayrılan miktar: 6
yapi2 için bellekte ayrılan miktar: 6
*/
```

13.10 Birlikler

Birlikler (**union**), Pascal dilindeki **record-case** talimatına benzer. Birlik, yapı (**struct**) gibi tanımlanır. Aralarındaki fark **yapı elemanlarının her birine ayrı bellek bölgesi ayrılırken, birlik üyelerinin her biri aynı bellek bölgesini paylaşırlar**. Birlik tanımına ilişkin sözde kod aşağıda verilmiştir;

```
union yapı-kimliği {
    veri-tipi yapı-elemanı-kimliği1;
    veri-tipi yapı-elemanı-kimliği1;
    ...
} [değişken-kimliği1][, değişken-kimliği2];
```

Bilgi

Birliğin bellek boyutu, en fazla bellek kaplayan elemanı kadardır.

Aşağıda üyelerinin aynı bellek bölgesini paylaştığını gösteren program örneği bulunmaktadır;

```
#include <stdio.h>
union tamsayiBirliğı {
    char baytlar[4];
    int  tamsayi;
    char bayt;
    short int kisatamsayi;
} birlik1,birlik2;
int main() {
    union tamsayiBirliğı birlik3,birlik4;
    birlik1.tamsayi=0x1B2C3D4F; //16lık sayılarda her çift rakam bir bayt olur
    printf("sizeof tamsayiBirliğı.baytlar: %d\n", sizeof birlik1.baytlar); //4
    printf("sizeof tamsayiBirliğı.bayt: %d\n", sizeof birlik1.bayt); //1
    printf("sizeof tamsayiBirliğı.tamsayi: %d\n", sizeof birlik1.tamsayi); //4
    printf("sizeof tamsayiBirliğı.kisatamsayi: %d\n", sizeof birlik1.kisatamsayi); //2
    printf("sizeof tamsayiBirliğı: %d\n", sizeof birlik1); //4
    printf("birlik1.tamsayi:%x\n",birlik1.tamsayi); //1b2c3d4f
    printf("birlik1.baytlar:%x-%x-%x-%x\n", birlik1.baytlar[0], birlik1.baytlar[1],
    ↪ birlik1.baytlar[2],birlik1.baytlar[3]); //4f-3d-2c-1b
    printf("birlik1.bayt:%x\n",birlik1.bayt); //4f
    printf("birlik1.kisatamsayi:%x\n",birlik1.kisatamsayi); //3d4f
    birlik1.bayt=0x00; //bayt değişkeni değiştirildiğinde diğer tüm üyelerin içeriği değişir!
    ↪ Tamsayının son iki hanesine dikkat ediniz!
    printf("birlik1.tamsayi:%x\n",birlik1.tamsayi); //1b2c3d00
    return 0;
}
```

13.11 Düşün ve Çöz

- ◊ **Düşün:** Yapı Dolgusu kavramını açıklayınız. İşlemcinin veriye erişim hızı ile bellek tasarrufu arasındaki denge bu noktada nasıl kurulur?
- ◊ **Düşün:** Birlik (**union**), neden aynı anda sadece bir üyenin değerini tutabilir? Bellek paylaşımı mantığını bir şema ile açıklayınız.
- ◊ **Düşün:** Bit alanları (**bit-fields**) kullanımı, düşük seviyeli donanım kontrolünde (**register**) neden vazgeçilmezdir?
- ◊ **Çöz:** Bir öğrencinin adını, numarasını ve 3 dersinin notunu tutan bir **struct** tanımlayınız. Bu yapıdan bir dizi oluşturup sınıf ortalamasını hesaplayınız.
- ◊ **Çöz:** İç içe yapılar (**nested structs**) kullanarak bir "Şirket" yapısı ve bu yapının içinde "Departman" ve "Çalışan" bilgilerini tutan bir model oluşturunuz.

- ◇ **Çöz:** Bir `union` kullanarak, 4 baytlık bir `int` sayının bellekteki her bir baytına karakter göstericisiyle erişen bir program yazınız.